VAULTX PROTOCOL • ETHEREUM SEPOLIA

A Decentralized Blockchain Finance Vault

Technical Architecture, Smart Contract Security & Production Web3 Implementation

Project Domain:	Decentralized Finance (DeFi) Self-Custody Vault
Blockchain Network:	Ethereum Sepolia Testnet (Chain ID: 11155111)
Smart Contract:	FinanceVault.sol (Solidity 0.8.24)
Live Contract Address:	**0xc105C3d4006d546D9eB92b72FEC2A1C18BcF027e**
Etherscan Verification:	https://sepolia.etherscan.io/address/0xc105C3d4006d546D9eB92b72FEC2A1C18BcF027e
GitHub Repository:	https://github.com/Hassaan1805/vaultX

1. Executive Summary & Problem Statement

Modern digital asset custody faces a critical dilemma between ease-of-use and cryptographic security. Centralized exchange vaults and poorly architected smart contracts frequently suffer from custodial freeze risks, reentrancy exploits, and administrative backdoors. **VAULTX** addresses these vulnerabilities by establishing a mathematically verifiable, 100% self-custodial digital asset escrow protocol.

Fundamental Protocol Safeguards:

- **Zero Custodial Intermediaries:** Users maintain sovereign ownership of private keys via MetaMask. No third party can freeze or confiscate assets.
- **Per-User Accounting Isolation:** Balances are tracked per msg.sender. User A cannot withdraw or affect funds belonging to User B.
- **Checks-Effects-Interactions (CEI):** Internal balance states are updated before external transfers occur, eradicating reentrancy drainage vectors.
- **OpenZeppelin ReentrancyGuard:** Both deposit and withdrawal execution are shielded by audited mutex locks.
- **Zero Privilege Levers:** Contains no onlyOwner functions, admin upgrade levers, or arbitrary withdrawal overrides.

2. System Architecture & Technology Stack

Component	Technology	Key Responsibilities
Smart Contract	Solidity 0.8.24	Core vault ledger, escrow accounting, reentrancy defense
Development Env	Hardhat 2.22.3	Compilation, Mocha/Chai unit testing, network deployment
Security Library	OpenZeppelin 5.0.2	Audited ReentrancyGuard mutex lock
Web3 Integration	ethers.js v6.12.1	BrowserProvider, contract calls, ABI encoding, event listeners
Client Framework	React 18 + Vite	Reactive state, fast HMR, optimized bundler
Styling System	Tailwind CSS	Fintech editorial theme, neon green (#C5FF36), glassmorphism
Network Node	Alchemy Sepolia RPC	High-speed EVM JSON-RPC provider on Ethereum Sepolia

Project File Architecture:

```
vaultX/
+-- contracts/
|   +-- contracts/
|   |   +-- FinanceVault.sol              # Core Escrow Contract (Solidity 0.8.24)
|   |   +-- mocks/ReentrancyAttacker.sol  # Security Attack Harness
|   +-- scripts/deploy.ts                 # Automated Sepolia Deployment & JSON Exporter
|   +-- test/FinanceVault.test.ts         # 12 Hardhat Unit Test Cases
|   +-- hardhat.config.ts                 # Hardhat Compiler & Network Config
+-- frontend/
|   +-- src/
|   |   +-- components/                   # Navbar, Hero, Stats, Cards, Activity, Security
|   |   +-- hooks/
|   |   |   +-- useWallet.ts              # MetaMask Provider, Network Detection & Polling
|   |   |   +-- useVaultContract.ts       # On-chain Read/Write & Real-Time Events
|   |   +-- lib/deployedContract.json     # Live Sepolia Contract Address & ABI
|   |   +-- App.tsx                       # Master Application Layout & Toast Coordinator
+-- .env                                  # Alchemy RPC & Testnet Deployer Private Key
+-- package.json                          # Unified Root Scripts (dev, build, test, deploy)
+-- README.md                             # Comprehensive Operations & Setup Manual
```

3. Smart Contract Implementation (`FinanceVault.sol`)

FinanceVault.sol is engineered to provide gas-optimized accounting, strict Checks-Effects-Interactions (CEI) guarantees, and clear custom errors for failure states.

A. State Storage Architecture:

```
// Active per-user liquid balances
mapping(address => uint256) public balances;

// Cumulative lifetime ETH deposited per user address
mapping(address => uint256) public userDeposited;

// Cumulative lifetime ETH withdrawn per user address
mapping(address => uint256) public userWithdrawn;

// Global lifetime cumulative statistics
uint256 public totalVaultDeposited;
uint256 public totalVaultWithdrawn;
```

B. Deposit Function (`deposit`):

```
function deposit() external payable nonReentrant {
    if (msg.value == 0) {
        revert ZeroAmount();
    }

    uint256 amount = msg.value;

    // State accounting (Effects)
    balances[msg.sender] += amount;
    userDeposited[msg.sender] += amount;
    totalVaultDeposited += amount;

    emit Deposit(msg.sender, amount, block.timestamp);
}
```

C. Withdrawal Function with Checks-Effects-Interactions (`withdraw`):

```
function withdraw(uint256 amount) external nonReentrant {
    // 1. CHECKS (Validation)
    if (amount == 0) {
        revert ZeroAmount();
    }
    uint256 currentBalance = balances[msg.sender];
    if (amount > currentBalance) {
        revert InsufficientBalance(amount, currentBalance);
    }

    // 2. EFFECTS (Internal state updated BEFORE external transfer)
    balances[msg.sender] = currentBalance - amount;
    userWithdrawn[msg.sender] += amount;
    totalVaultWithdrawn += amount;

    emit Withdrawal(msg.sender, amount, block.timestamp);

    // 3. INTERACTIONS (Safe native low-level transfer)
    (bool success, ) = msg.sender.call{value: amount}("");
    if (!success) {
        revert TransferFailed();
    }
}
```

D. Direct Ether Transfer Fallback (`receive`):

```
receive() external payable {
    if (msg.value == 0) {
        revert ZeroAmount();
    }
    uint256 amount = msg.value;
    balances[msg.sender] += amount;
    userDeposited[msg.sender] += amount;
    totalVaultDeposited += amount;
    emit Deposit(msg.sender, amount, block.timestamp);
}
```

4. Automated Unit Testing & Security Verification

All 12 automated unit tests pass in Hardhat. The test suite verifies deposit state accumulation, overdraft protection, direct transfers, multi-user isolation, and reentrancy attack resistance via ReentrancyAttacker.sol:

Test Module	Test Assertion Description	Status
1. Init	Verifies initial TVL, balances, and cumulative counters are 0.	**PASSED**
2. Deposit	Verifies balance updates and event emissions on single/multiple deposits.	**PASSED**
2. Zero Dep	Asserts revert with custom error ZeroAmount() on 0 ETH deposit.	**PASSED**
2. Fallback	Verifies plain Ether transfers trigger receive() and credit balance.	**PASSED**
3. Partial With	Verifies partial withdrawal updates balance and transfers correct Ether.	**PASSED**
3. Full With	Verifies user can withdraw 100% of their deposited balance safely.	**PASSED**
3. Zero With	Asserts revert with custom error ZeroAmount() on 0 ETH withdrawal.	**PASSED**
3. Overdraft	Asserts revert with InsufficientBalance(requested, available).	**PASSED**
4. User Isolation	Guarantees User A cannot withdraw User B's funds even if contract has TVL.	**PASSED**
4. Account Stats	Verifies getUserStats() returns correct 3-tuple metrics.	**PASSED**
5. Reentrancy	Malicious contract attempting recursive callback fails and reverts safely.	**PASSED**

Reentrancy Defense Test Code:

```
it("should defend against reentrancy attacks via ReentrancyGuard", async function () {
    // User1 deposits 5 ETH into the vault
    await vault.connect(user1).deposit({ value: ethers.parseEther("5.0") });

    // Deploy malicious attacker contract targeting vault
    const AttackerFactory = await ethers.getContractFactory("ReentrancyAttacker");
    const attacker = await AttackerFactory.connect(attackerUser).deploy(await vault.getAddress());

    // Attacker attempts recursive reentrant drain -> Reverts!
    await expect(
        attacker.connect(attackerUser).initiateAttack({ value: ONE_ETH })
    ).to.be.reverted;

    // Vault TVL and User1 funds remain 100% intact
    expect(await vault.getUserBalance(user1.address)).to.equal(ethers.parseEther("5.0"));
    expect(await vault.getContractBalance()).to.equal(ethers.parseEther("5.0"));
});
```

5. Frontend Web3 Integration Engine

The frontend application provides seamless synchronization between MetaMask browser wallets, Ethereum Sepolia RPC nodes, and the FinanceVault contract. Key innovations include:

A. Dual-Source Balance Resolution (`useWallet.ts`):

MetaMask multi-chain UI frequently displays tokens across different chains while keeping the active connection on Ethereum Mainnet. To prevent confusion where a user holds 0.0500 SepoliaETH but sees 0.0000 ETH on Mainnet, VAULTX implements dual-source balance querying. It queries the active MetaMask provider while simultaneously checking the public Sepolia RPC directly:

```
// 1. Fetch from active MetaMask provider directly
const hexBal = await window.ethereum.request({
    method: 'eth_getBalance',
    params: [account, 'latest'],
});
if (hexBal) activeRaw = BigInt(hexBal);

// 2. Fetch directly from Sepolia public RPC for guaranteed testnet accuracy
const sepProvider = new ethers.JsonRpcProvider('https://ethereum-sepolia-rpc.publicnode.com');
sepoliaRaw = await sepProvider.getBalance(account);

// If MetaMask is on Mainnet, the dApp still knows the user has 0.0500 SepoliaETH
// and renders an instant 1-click 'Switch to Sepolia' action!
```

B. Gas Reserve Buffer Protection (`calculatePercentageAmount`):

When depositing, clicking 100% (MAX) in naive dApps drains all Ether, causing MetaMask to throw an *'insufficient funds for gas'* error. VAULTX calculates a dynamic 0.003 ETH buffer reserve automatically:

```
export function calculatePercentageAmount(rawBalance: bigint, percentage: number, isDeposit: boolean = true) {
    if (rawBalance <= 0n) return '0';
    if (isDeposit && percentage === 100) {
        const gasBuffer = ethers.parseEther('0.003'); // Reserve gas buffer
        if (rawBalance <= gasBuffer) return '0';
        return ethers.formatEther(rawBalance - gasBuffer);
    }
    const fraction = BigInt(percentage);
    return ethers.formatEther((rawBalance * fraction) / 100n);
}
```

C. Real-Time On-Chain Event Indexing (`useVaultContract.ts`):

```
// Live On-Chain Event Subscriptions
contract.on('Deposit', (user, amount, timestamp, event) => {
    fetchStats(); // Instantly update balances
    addNewTransactionRecord({ type: 'deposit', user, amount, timestamp, txHash: event.log.transactionHash });
});

contract.on('Withdrawal', (user, amount, timestamp, event) => {
    fetchStats();
    addNewTransactionRecord({ type: 'withdrawal', user, amount, timestamp, txHash: event.log.transactionHash });
});
```

6. Operational Deployment & Execution Guide

Follow these concise terminal commands to run, test, and deploy the project:

Step 1: Install Dependencies

npm --prefix contracts install
npm --prefix frontend install

Step 2: Execute Unit Tests

npm --prefix contracts test (All 12 unit tests execute and pass in ~1 second)

Step 3: Deploy to Ethereum Sepolia

Ensure .env contains your SEPOLIA_RPC_URL and PRIVATE_KEY, then run:
npm --prefix contracts run deploy:sepolia

Notice: The deployment script automatically updates frontend/src/lib/deployedContract.json with the newly generated contract address and ABI!

Step 4: Launch Frontend Interface

npm run dev (Launches Vite development server on **http://localhost:3000**)

7. Conclusion & Engineering Significance

VAULTX successfully demonstrates a production-grade decentralized finance application adhering to the highest standards of Web3 engineering: checks-effects-interactions state mutation, OpenZeppelin reentrancy protection, zero administrative custody vulnerabilities, and a high-contrast editorial fintech UI with dual-source real-time blockchain balance synchronization. The project serves as a comprehensive portfolio showcase for academic viva presentations and production smart contract design.